Major in Artificial Intelligence

Artificial intelligence is an increasingly important technology that affects many disciplines, including science, engineering, business and public policy. Applications of artificial intelligence include machine learning, computer vision, natural language processing, robotics, data analysis and automated decision systems. Graduates with training in artificial intelligence work in fields such as data science, machine learning engineering, robotics, intelligent software development and research.

Artificial intelligence majors at Christopher Newport University study the computational, mathematical and scientific foundations required to design and deploy intelligent systems. Core courses provide a background in computer science, programming, mathematics and scientific computing. Advanced courses in the program focus on machine learning, neural networks, computer vision, large language models, data science and the engineering tools used to deploy modern AI systems.

Students also develop practical experience working with real datasets, training and evaluating machine learning models, building AI pipelines and deploying systems in cloud and high-performance computing environments. The program culminates in a capstone project in which students design and implement an original artificial intelligence system.

Graduates will be prepared for employment in artificial intelligence, data science and software development, or for further study in graduate programs in artificial intelligence, computer science or related fields.

Students pursuing the major in artificial intelligence are strongly encouraged to pursue studies in complementary academic areas where intelligent systems are widely applied. Minors or coursework in fields such as mathematics, physics, business, economics, cybersecurity or the natural sciences may provide valuable interdisciplinary perspectives.

Degree requirements are those of the Bachelor of Science degree. In addition to completing the liberal learning curriculum, the artificial intelligence major requires successful completion of the following courses with no more than two grades below C-.

1. Programming and Computer Science Core:

CPSC 150/150L, 250/250L, 255, 270, 327, 410, 420.

2. Mathematics and Computational Foundations:

MATH 125, 140, 240, 250, 260, 335; ENGR 213.

3. Science and Experimental Methods:

PHYS 151/151L–152/152L or PHYS 201/201L–202/202L.

4. Artificial Intelligence and Data Science Core:

DATA 201, DATA 301, CPSC 336, CPSC 401, CPSC 403.

5. Supporting Courses:

CPSC 440; CPEN 371W

6. Major Elective:

Select two from: CPSC 441, CPSC 472, CYBR 484, CPSC 510, PHYS 441/541, MATH 380, or other approved artificial intelligence electives.

7. Capstone:

CPSC 497 (Capstone Project in Artificial Intelligence).

The recommended sequence of courses integrates programming, mathematical foundations, machine learning, neural networks and large language models over four years to prepare students for modern artificial intelligence development workflows.

Useful Course Descriptions :

DATA 201. Introduction to Data Science

This course spends half of the time on data analysis/visualization and the second half is intro ML

- Python review: Data types, Text IO, Numpy operations, Matplotlib
- Python & tools (Anaconda, Jupyter)
    - No conda environments or venv at this point
    - Generative AI tools are actively encouraged for biweekly projects (50% of the grade) but not allowed on exams which have very simple problems. Must follow Elsevier rules for AI use.
- Data formats, data wrangling, time-series data
- Pandas Series, Dataframe introduction. Basic operations
- Data Visualization (Chapter's 5&9 from Edward Tufte): Minimalism, Data-Ink Ratio, No chart junk
- Classification with kNN using the Iris Dataset, write kNN algorithm in numpy then use sklearn.
    - Feature normalization discussed
- Nonlinear Least Squares Regression (scipy.curve_fit, fit parameters with uncertainties), Covariance Matrix, Initial parameters, failed fits
- Overfitting and underfitting with polynomials, ties in later with NN
- Neural Network Introduction, History, AI Winters, Perceptron. Uses Tensorflow
- Activation functions: Sigmoid, relu
- MNIST Handwritten digit classifier with MLP and CNN
- CIFAR 10 and 100 classification with CNNs
- Using pretrained models from hugging face (YOLO models in real time with student's webcams and VDOT cameras)
    - Image segmentation, object recognition

DATA 301 – Data Science Methodology

This course is focused on ML with structured data, python and pandas. It includes:

- Python environments & tools (Anaconda, Jupyter)
- General Approach to a Data Science Question
- Cleaning Data (handling duplicates & NaNs, text cleanup, handling categorical data)
- Feature Scaling & Dimensionality Reduction (PCA, t-SNE, UMAP)
- Pandas Pipelines
- Data Sampling (for large datasets to speed development)
- Non Supervised Learning with K-Means DBSCAN & HDBSCAN Clustering Algorithms
- Elbow/Silhouette Methods
- Supervised Learning

- Dataset train/test split
- Bias/Variance tradeoff
- Regression
- Decision Trees
- Random Forests
- Cross-Validation & Hyperparameter Tuning
- Precision/Recall, F1 Score, Confusion Matrix
- Explainable AI Concepts (feature importance, permutation importance, PDP, ICE plots, SHAP values)
- Gradient Boosted Trees
- Time Series Analysis

CPSC 401: Neural Foundations & Computer Vision (3 Credits)

Introduces students to the perception layer of modern AI systems through hands-on development of deep learning models for visual tasks. Using PyTorch as the primary framework, students learn to construct and train neural networks beginning with tensor operations, autograd, and modern optimization techniques before progressing to convolutional neural networks for spatial feature extraction. The course examines contemporary deep architectures such as ResNet and emphasizes transfer learning using pre-trained models from the *timm* ecosystem. Students then apply these methods to real-world computer vision problems, including training and evaluating object detection systems based on the YOLO architecture and working with practical evaluation metrics such as Intersection over Union and mean Average Precision. The course concludes with segmentation methods such as U-Net and an introduction to exporting trained models to ONNX for deployment, preparing students for subsequent work on hardware optimization and edge AI systems.

Week	Topic	Content & Key Skills
1-2	**Backprop**	Review of partial diff, chain rule. Intro to backprop.
3-4	**The PyTorch Ecosystem**	**Tensors & Autograd:** Deep dive into PyTorch tensors, GPU acceleration, and computational graphs. **Foundations:** Building an MLP directly in PyTorch. Understanding modern loss functions (CrossEntropy, Focal Loss) and optimizers (SGD with

Week	Topic	Content & Key Skills
		momentum, AdamW).
5-7	**Convolutional Neural Networks (CNNs)**	**Spatial Feature Extraction:** The math and intuition of convolutions, pooling, stride, and padding. **Training Pipelines:** Building early ConvNets. Mastering PyTorch Dataset and DataLoader classes. **Robustness:** Implementing data augmentation strategies and handling class imbalance in training data.
8–10	**Modern Architectures & Transfer Learning**	**Deep Networks:** Understanding the vanishing gradient problem and how residual connections (ResNet) solve it. **The 'timm' Library:** Accessing state-of-the-art pre-trained weights. **Fine-Tuning:** Freezing layers, replacing classification heads, and customizing loss functions for highly specialized, domain-specific tasks.
11-12	**Real-Time Object Detection**	**Beyond Classification:** Bounding boxes, anchor boxes, and the YOLO (You Only Look Once) architecture. **Evaluation:** Mastering rigorous evaluation metrics, specifically mean Average Precision (mAP) and Intersection over Union ($IoU = \frac{
13–14	**Segmentation & Edge Readiness**	**Pixel-Level Prediction:** Semantic and instance segmentation using U-Net architectures, crucial for medical or industrial defect imaging.

CPSC 403: Large Language Models & Agentic Systems (3 credits)

Examines the architecture and practical deployment of modern language models, beginning with the transformer framework that underlies contemporary AI systems. Students study the mathematics of attention and implement simplified encoder–decoder transformers before transitioning to modern development workflows using the Hugging Face ecosystem. The course emphasizes practical model adaptation under real hardware constraints through parameter-efficient fine-tuning methods such as LoRA and QLoRA applied to open-weight models. Students then build retrieval-augmented generation (RAG) pipelines using vector embeddings, semantic search, and vector databases to integrate external knowledge with language models. The final portion of the course focuses on agentic AI systems, where models interact with tools and external environments through function calling and structured reasoning frameworks to execute multi-step tasks and autonomous workflows.

Week	Topic	Content & Key Skills
1–3	**Transformers**	**The Attention Mechanism:** Why RNNs/LSTMs hit a wall. The mathematics of Scaled Dot-Product Attention: $$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$ **Architecture:** Deep dive into the "Attention is All You Need" paper.
3-5	**The Hugging Face Ecosystem & PEFT/LORA**	Hugging Face transformers and datasets libraries. Parameter-Efficient Fine-Tuning (PEFT) and Low-Rank Adaptation (LoRA/QLoRA). **Lab:** Fine-tuning an open-weight model (e.g., Llama 3 or Mistral) on technical manuals or scientific papers (a great tie-in to JLab).
6-7	**Reinforcement Learning Basics**	How models learn behavior from feedback rather than labels
8–10	**Retrieval-Augmented Generation (RAG)**	**The Modern Data Stack:** The mathematics of vector embeddings and cosine similarity. Setting up Vector Databases (ChromaDB). **Ingestion & Retrieval:** Document parsing, chunking strategies (recursive vs. semantic), and basic semantic search. **Advanced RAG:** Improving context quality. Implementing bi-encoders for embedding and cross-encoders for re-ranking.

Week	Topic	Content & Key Skills
11–14	**Agentic Workflows & Tool Calling**	**Beyond Text Generation:** Teaching LLMs to interact with environments. The ReAct (Reasoning and Acting) framework. **Function Calling:** Equipping LLMs with external "tools" (calculators, SQL executors, Python REPLs, web search). **Multi-Step Reasoning:** Building autonomous pipelines that can break down complex, multi-step prompts into actionable plans, execute them, and evaluate the results.

MATH 125. Elementary Statistics (3-3-0) LLFM This course is a general survey of descriptive and inferential statistics. Topics include descriptive analysis of univariate and bivariate data, probability, standard distributions, sampling, estimation, hypothesis testing and linear regression. Students may not receive credit for this course after receiving a grade of C- or higher in MATH 435.

Here is a MATH 125 list of course topics:
Course introduction and expectations
Terminology, Sampling, §1.1-1.2
Terminology (continued) §1.3
Frequency Distributions, Histograms, §2.1
Measures of Center and Measures of Variation, §3.1-3.2
Quartiles and Boxplots, §3.3
Scatter Diagrams and Linear Correlation, §4.1
Linear Regression, §4.2
Review for Test 1
Test 1, §1.1-1.3, 2.1, 3.1-3.3, 4.1-4.2
Probability: Probability Basics, Addition Rule, Multiplication Rule, §5.1
Contingency Tables, §5.2

Trees and Counting Techniques, §5.3
Random Variables, Discrete Probability Distributions, Expected Value, §6.1
Binomial Probability Distribution, §6.2
Continuous Probability Distributions, §7.1
Standard Normal Distribution, §7.2
Assessing Normality, §7.3
Review for Test 2
Test 2, §5.1-3, 6.1-3, 7.1-3
Sampling Distributions, §7.4
The Central Limit Theorem, §7.5
Normal Approximation, §7.6
Estimating μ when σ is Known, §8.1
Estimating μ when σ is Unknown, §8.2
Estimating p in the Binomial Distribution, §8.3
Review for Test 3
Test 3, §7.4-7.6, 8.1-8.3
Introduction to Statistical Tests, §9.1
Testing the Mean μ, §9.2
Testing a Proportion p, §9.3
Testing σ 2 or σ, §11.3
Review for Test 4
Test 4, §9.1-9.3, 11.3
Review
Review
Final Exam, 8-10:30 am, cumulative

MATH 335. Applied Probability (3-3-0) Prerequisite: MATH 240 with a C- or higher. Fall. Elementary probability theory including combinatorics, distributions of random variables, conditional probability and moment generating functions. An introduction to stochastic processes including such topics as Markov chains, random walks and queuing theory. Case studies may be considered. Computer projects may be required. typical core topics look roughly like this:

- Probability spaces
- Conditional probabiolity and independence
- Common probability distributions
- Discrete spaces vs. continuous spaces

- Random variables
- Sums of random variables
- Covergence and limit theorems

One commonly used text is the open-source Probability: Lectures and Labs by Mark Huber.

ENGR 213. Discrete Structures for Computer Applications (3-3-0) Prerequisite: CPSC 150. Fundamental mathematical tools used in the analysis of algorithms and data structures, including logic, sets and functions, recursive algorithms and recurrence relations, combinatorics and graphs

CPSC 441. Big Data Technologies (3-3-0) Prerequisites: MATH 235 or 260; CPSC 336, 440; restricted to BSIS majors or permission of department chair. This course covers facets of cloud computing and big data management, including the study of the architecture of the cloud computing model with respect to virtualization, multitenacy, privacy, security, cloud data management and indexing, scheming and cost analysis; it also includes relevant programming models, crowd sourcing and data provenance.

MATH 140. Calculus and Analytic Geometry (4-4-1) LLFM Prerequisite: MATH 130 or 132 with a C- or higher (MATH 132 is preferred) or an acceptable score on the Calculus Readiness Assessment. An introduction to the calculus of elementary functions, continuity, derivatives, methods of differentiation, the Mean ValueTheorem, curve sketching, applications ofthe derivative, the definite integral, the Fundamental Theorems of Calculus, indefinite integrals and logarithmic and exponentialfunctions. The software package Mathematica will be used.

MATH 240. Intermediate Calculus (4-4-0) Prerequisite: MATH 140 or 148 with a C- or higher. Techniques of integration, L'Hospital's Rule, application of integration, approximations, Taylor's Theorem, sequences and limits, series of numbers and functions, power series and Taylor series. The software package Mathematica will be used.

MATH 250. Multivariable Calculus (3-3-0) Prerequisite: MATH 240 with a C- or higher. An introduction to the calculus of real-valued functions of more than one variable. The geometry of three-space, vectorvalued functions, partial and directional derivatives, multiple and iterated integrals and applications. The software package Mathematica will be used.

MATH 260. Linear Algebra (3-3-0) Prerequisite: MATH 240 or MATH 245 with a C- or higher. Systems of linear equations, matrix operations, determinants, vectors and vector spaces, independence, bases and dimension, coordinates, linear transformations and matrices, eigenvalues and eigenvectors.

CPSC 510. Artificial Intelligence I (3-3-0) (Fall) . This course is an introduction to the mathematical and computational foundations of artificial intelligence. Its emphasis is on those elements of artificial intelligence that are most useful for practical applications. Topics include heuristic search, problem solving, game playing, knowledge representation, logical inference, planning, reasoning under uncertainty, expert systems, machine learning and language understanding. Programming assignments are required.

Includes Markov Decision Processes (MDP) and Reinforcement Learning, Hidden Markov Models, Bayes nets and Naïve Bayes

CPSC 150. Introduction to Programming (3-3-0) LLFR Pre or Corequisite: CPSC 150L This course is an introduction to problem solving and programming. Topics include using primitive and object types, defining Boolean and arithmetic expressions, using selection and iterative statements, defining and using methods, defining classes, creating objects and manipulating arrays. Emphasis is placed on designing, coding and testing programs using the above topics. Satisfies the logical reasoning foundation requirement.

CPSC 150L. Introduction to Programming Laboratory (1-0-3) Pre or Corequisite: CPSC 150. Laboratory course supports the concepts in CPSC 150 lecture with hands-on programming activities and language specific implementation. Laboratory exercises stress sound design principles, programming style, documentation and debugging techniques. Lab fees apply each term.

CPSC 250. Programming for Data Manipulation (3-3-0) Prerequisites: Grade of C- or higher in CPSC 150/150L Corequisites: CPSC 250L and MATH 135 or 140 or 148 or permission of department chair This course builds upon concepts taught in CPSC 150 and provides continuing study of data storage and manipulation, and introduces their application to scientific computing and visualization. Specific topics include object oriented design, programming style, debugging and algorithm design. The course will incorporate the use of existing libraries for data processing and visualization.

CPSC 250L. Programming for Data Manipulation Laboratory (1-0-3) Prerequisites: CPSC 150/150L with a grade of C- or higher. Pre or Corequisite: CPSC 250. Laboratory course supports the concepts in CPSC 250 lecture with hands-on programming activities and language specific implementation. Laboratory exercises stress sound design principles,

programming style, documentation and debugging techniques. Lab fees apply each term.

CPSC 255. Programming for Applications (3-3-0) Prerequisite: CPSC 250 with grade of C- or higher or consent of instructor. This course provides a study of programming and problem solving using a scalable structured programming language. The course assumes competency with programming and problem solving using variables, conditional statements, loops, classes (objects) and arrays. The course begins with a brief introduction to these prior programming concepts, before moving on to more advanced concepts such as class inheritance and interfaces, generics and beginning data structures such as linked lists, stacks and queues. The course includes more advanced programming techniques such as exceptions, recursion, networking and data structures.

CPSC 270. Data and File Structures (3-3-0) Prerequisite: CPSC 255 with a grade of C- or higher. Pre or Corequisite: ENGR 213. Study of objects and data structures. Trees, graphs, heaps with performance analysis or related algorithms. Structure, search, sort/merge and retrieval of external files. Programming assignments will involve application of the topics covered.

CPSC 327. C++ Programming (3-3-0) [Formerly CPSC427, equivalent] Prerequisites: ENGR 213 and a grade of C- or higher in CPSC 255. Designed for students who already know how to program, but do not know C++. This is a comprehensive introduction to C++. The course will emphasize basic C++, in particular memory management, inheritance and features needed for low level programming.

CPSC 410. Operating Systems I (3-3-0) Prerequisites: CPSC 270 and 327. Introduction to operating systems, I/O processing, interrupt structure and multiprocessing-multiprogramming, job management, resource management, batch and interactive processing, memory management, multithreaded programming in C++, deadlock problems, thread synchronization. ½ of the course is multithreaded systems, atomics, mutexes, races, condition variables, semaphores.

CPSC 420. Algorithms (3-3-0) Prerequisites: CPSC 270 and ENGR 213. The application of analysis and design techniques to numerical and non-numerical algorithms which act on data structures. Examples will be taken from areas such as combinatorics, numerical analysis, systems programming and artificial intelligence.

CPSC 440. Database Management Systems (3-3-0) Prerequisites: CPSC 250 and 250L. Database (DB) concepts. Relational, hierarchical and network models. Query languages, data sub-languages and schema representations. The DB environment: DB administration, security, dictionaries, integrity, backup and recovery. May be taken as research intensive.

CYBR 484: AI Applications in Cybersecurity: This course examines the use of artificial intelligence to analyze and defend modern computing systems. Students learn to treat network traffic, system logs, and executable files as datasets that can be analyzed with statistical and machine learning methods. Using Python, students build tools for parsing security data, detecting anomalous network activity, classifying malware, and identifying phishing attempts. The course also introduces techniques for securing AI-enabled systems, including defenses against prompt injection and related attacks on automated decision systems. Emphasis is placed on practical workflows for threat detection, investigation, and risk prioritization within complex computing environments.

CPSC 472. Introduction to Robotics (3-3-0) Prerequisites: CPSC 255 or 256 and MATH 235 or 260 or ENGR 210 or PHYS 340 each with a grade of C- or higher. This course presents an overview of applied robotics. The course will cover introductions to configuration space representations, rigid body transforms in 2D and 3D, robot kinematics, basic control theory, motion planning, perception and machine decision making. Perception topics include basic computer vision and laser rangefinder (LIDAR)-based obstacle detection and mapping. The course includes hands on development and system integration using various robotic platforms. Programming will be done in Ubuntu Linux in a mixture of C++ and Python; no prior experience is required, butstudents will be expected to self-teach the specifics necessary to complete the projects.

CPSC 497 Capstone Project in Artificial Intelligence: In this course, you will apply machine learning, data engineering, and algorithmic foundations to propose, architect, and deploy a semester-long artificial intelligence project. The project requires a computationally significant effort, integrating data pipeline construction, model selection and optimization (e.g., hyperparameter tuning or fine-tuning), and rigorous performance validation. The course follows a structured lifecycle, beginning with selecting and defending a technical project proposal. Once approved, weekly status updates are expected alongside two intermediate presentations. This process culminates in a final project submission, defense, and poster presentation.

CPSC 441. Big Data Technologies (3-3-0) Prerequisites: MATH 235 or 260; CPSC 336, 440; restricted to BSIS majors or permission of department chair. This course covers facets of cloud computing and big data management, including the study of the architecture of the cloud computing model with respect to virtualization, multitenacy, privacy, security, cloud data management and indexing, scheming and cost analysis; it also includes relevant programming models, crowd sourcing and data provenance.

CPEN 371. WI: Computer Ethics (2-2-0) Prerequisites: ENGL 223 with a C- or higher; major or minor in PCSE. This course covers contemporary ethical issues in science and engineering. A framework for professional activity is developed, which involves considerations and decisions of social impact. Current examples will be studied, discussed and reported: IEEE and ACM codes of ethics, software and hardware property law, privacy, social implications of computers, responsibility and liabilities and computer crime.

The following course, CPSC 336 is going to have its name and content description changed to something similar to:

CPSC 336. Cloud, Linux, and Network Administration
Study of Linux-based network and cloud systems, with emphasis on command-line administration, TCP/IP networking, secure remote access, server configuration, virtualization, cloud compute, storage, and database services. Students configure and administer systems

such as web servers, database servers, virtual machines, and cloud instances, with attention to functionality, reliability, and security.